

Lethe: A Core Calculus of Mergeable Forgetting

A pen-and-paper kernel for building efficient MRDTs by construction (the oq:proglogic design note)

Sal metatheory note
FP Launchpad, IIT Madras

July 9, 2026

Abstract

Every efficient mergeable replicated data type (MRDT) in the Sal catalogue is a *forgetful image* of a free, remember-everything datatype. Lethe makes the forgetting an operator. A term of the calculus denotes an MRDT together with the retraction exhibiting it as an image of the free event log; the typing derivation assembles that retraction compositionally, and each constructor owes a fixed, leaf-level proof obligation. Four consequences reorganize the existing framework rather than extending it: the LCA of three-way merge is *the price of forgetting*; efficiency is its *yield*; the conditioned verification-condition bundle is the *proof obligation* of the **Forget** operator, whose causal-delta law CDVC3 is already machine-checked to be independent; and the observational equivalence $\approx$ the framework reasons up to is the *congruence* of **Forget**. The assertion layer is separation logic over the *id-heap*: states decompose as separating conjunctions over identities, operations have footprints, conflict resolution is a per-atom overlap policy, and the honesty conditions of the hardest datatype — the tombstone-free RGA’s ancestor paths — are list-segment assertions carried by operations as birth certificates. No global rely appears in the judgment.

Status. This is a pen-and-paper design note; nothing in it is mechanized. Every claim is either backed by an existing kernel-checked artifact (cited inline as **code**) or explicitly marked as an obligation, a conjecture, or an open question.

Remark 0.1 (Naming). *Lethe is the river of forgetting. The calculus has one non-trivial operator, Forget; the LCA is the price of forgetting; and the soundness theorem (§8) — the efficient term is observationally equivalent to its free, remember-everything expansion — is an un-forgetting theorem, so it is named **Aletheia** (a-lēthē: truth as un-forgetting). The LCA-recovery law of level F2 (§6) is **Eunoë**, Lethe’s paired river of restored memory.*

Contents

1	Thesis	2
2	Preliminaries	2
3	The Free Object: the Log	3
4	The Id-Heap and the Assertion Layer	4

5	The Kernel	5
5.1	Grammar	5
5.2	The typing judgment	5
5.3	Rules	5
6	The Forgetting Ladder	6
7	Worked Derivations	8
7.1	LWW register — $\text{Forget}[\text{flat}]_{\text{keep-max}}(\text{GSet } (\mathbb{N} \times \mathbb{N} \times \tau))$ [F1]	8
7.2	Counter — $\text{Forget}[\text{flat}]_{\text{card}}(\text{GSet } (ts \times rid))$ [F2]	9
7.3	Efficient OR-set — $\text{Forget}[\text{cc}]_{\text{live}}(\otimes_{\text{inst}}(\text{GSet } \mathbf{1} \otimes \text{GSet } \mathbf{1}))^{\text{add-wins}}$ [F2]	9
7.4	Tombstone-free RGA — $\text{Forget}[\text{cc}]_{\text{drop+rehome}; \nu_{\text{op}}}(\text{tombstoned RGA})$ [F3]	10
7.5	Guarded exemplars, briefly	10
7.6	The catalogue as Lethe terms (proposed; validation = RQ1)	11
8	Aletheia: the Soundness Theorem (Statement Owed)	11
9	What the Calculus Predicts and Refuses	12
10	Open Questions	13
11	Relation to Existing Artifacts (crosswalk)	13

1 Thesis

Every efficient mergeable replicated data type is a forgetful image of a free, remember-everything datatype. Lethe makes the forgetting an operator. Four consequences, each of which reorganizes an existing piece of the framework rather than adding a new one:

1. **The LCA is the price of forgetting.** A datatype that remembers everything merges by union and never needs a common ancestor. Three-way merge exists because the normal form has discarded information that the join of two branches requires; the LCA is where it is recovered.
2. **Efficiency is the yield of forgetting.** “Efficient OR-set”, “closed-form counter”, “tombstone-free RGA” are all the same move: a retraction onto a normal form.
3. **The conditioned VC bundle is the proof obligation of Forget.** The causal-delta axiom CDVC3 — machine-checked independent (`Refutations/CD_Not_Derivable_Ternary.lean`; see `CD_MINIMALITY_TERNARY.md`) — is the primitive law of level-F2 forgetting.
4. **$\approx$ is the congruence of Forget.** The observational equivalence the framework reasons up to is not an artifact; it is the statement that forgetting is sound.

2 Preliminaries

This section is self-contained; no companion note is assumed.

Operations and events. Fix a type A of application operations. An *event* is $\text{Op } A := \mathbb{N} \times \mathbb{N} \times A$ — a globally unique timestamp, a replica id, and the payload. Timestamps are unique across the execution (`distinct_ops`).

MRDT signature. An MRDT is $D = \langle \text{State}, \text{init}, \text{do}, \text{mergeL}, \text{rc} \rangle$ where $\text{do} : \text{State} \rightarrow \text{Op } A \rightarrow \text{State}$, $\text{mergeL} : \text{State} \rightarrow \text{State} \rightarrow \text{State} \rightarrow \text{State}$ is the three-way merge (first argument the LCA), and $\text{rc} : \text{Op } A \rightarrow \text{Op } A \rightarrow \{\text{fst_then_snd}, \text{snd_then_fst}, \text{Either}\}$ arbitrates non-commuting concurrent pairs. Binary merge, when it exists, is the slice merge $a b = \text{mergeL init } a b$.

Version DAG. Replicas evolve by local steps (1-parent commits, apply `do`) and merge steps (2-parent commits, apply `mergeL` at the DAG-LCA of the parents). A *configuration* is coherent when every merge’s LCA argument is the fold of the shared causal prefix of its parents.

Remark 2.1 (Load-bearing caution). *In a general version DAG two branches may share events beyond their LCA (the same operation delivered to both sides via different merges), so no per-triple state equation of the form $\nu(A \sqcup B) = m(\nu L, \nu A, \nu B)$ can be a definition of correctness. This is exactly why the framework’s contract is a DAG-inductive VC bundle and why no lattice-flavoured contract can exist (`mrdr-metatheory.pdf`, delta-contract section). Lethe inherits this: the F2 obligation (§6) is stated per reachable configuration, not per triple.*

The judgment. D is *RA-linearizable up to $\approx$* when at every version of every reachable coherent configuration, the state is $\approx$ -equal to the fold (`do` over `init`) of some linearization of the version’s visible events that respects causal order and `rc`. Here $\approx$ is an equivalence on `State`, congruent for `do` and `mergeL`, under which queries are invariant. For datatypes whose operations are only meaningful on some states, the judgment is conditioned on *honest delivery*: every `op` is applicable at each state it is applied to, and *accurate at birth* (its payload correctly describes the issuing state). Only level F3 and `Guard` consume honesty; F0–F2 derivations are honesty-free.

The metatheorem interface. The framework provides: (i) the flat ternary metatheorem — `CoreVCs3CD + FeasibleDeltaVCs3 + CDVC3  $\Rightarrow$  JoinLemma3  $\Rightarrow$  RA-lin` (eight VCs, validated, minimal at `CDVC3`); (ii) the conditioned metatheorem consuming honest delivery (the tombstone-free RGA’s route, `MRDT_Instances/RGA/RA_Lin.lean`); (iii) the product kit (`Metatheory/Product*.lean`): convergence by concatenation witness, one-sided safety, and the $\approx$ -quotient lift. Lethe treats these as the semantic soundness lemmas its typing rules will invoke; this note adds no new metatheory.

3 The Free Object: the Log

Definition 3.1 (The log).

$$\begin{array}{ll} \text{Log}(A) : & \text{State} := \text{Finset } (\text{Op } A) \qquad \text{init} := \emptyset \\ & \text{do } s \ o := \{o\} \cup s \qquad \text{mergeL } l \ a \ b := a \cup b \quad (l \text{ unused: remembering is free}) \\ & \text{rc } _ _ := \text{Either} \qquad \approx := (=) \end{array}$$

On coherent configurations $l \subseteq a \cap b$, so union is the pushout of the two branches over the LCA; every linearization folds to the same union, and RA-linearizability is trivial. $\text{Log}(A)$ is *free* in the operative sense:

For any MRDT D over the same operations, RA-linearizability of D says exactly that every reachable state of D is ($\approx$ to) the image of a reachable Log-state under D 's own fold. **Correct = a forgetful image of the log.**

The catalogue proves this datatype-by-datatype. Lethe's job is to build the images *compositionally*: a term denotes a datatype together with the retraction exhibiting it as an image, and the typing derivation assembles the global retraction from per-constructor pieces.

4 The Id-Heap and the Assertion Layer

Id-heap. A Lethe state is presented as a separating conjunction of cells indexed by keys (element identities, timestamps, replica ids, unit for singleton cells):

$$\sigma \models \bigotimes_{k \in K} \text{cell}_k.$$

Assertions.

$$P ::= \text{emp} \mid k \mapsto v \mid P * P \mid \text{lseg}(k_0 \cdots k_n) \mid \text{tok}(n) \mid \ulcorner \varphi \urcorner$$

$k \mapsto v$ — key k holds cell value v ; $*$ — disjoint composition; $\text{lseg}(k_0 \cdots k_n)$ — a chain $k_0 \mapsto (_, k_1) * \cdots * k_{n-1} \mapsto (_, k_n)$ in a parent-pointer cell family (the RGA ancestor path); $\text{tok}(n)$ — n escrow tokens (the bounded-counter / budget discipline); $\ulcorner \varphi \urcorner$ — pure facts.

Footprints and locality. Each operation has a footprint $\text{fp}(o) \subseteq K$ — the keys its **do** reads or writes.

Proposition 4.1 (Locality principle). $\text{fp}(o_1) \cap \text{fp}(o_2) = \emptyset$ implies o_1, o_2 commute on all states, and $\text{rc}(o_1, o_2) = \text{Either}$ is forced.

Consequently rc is not a global relation but a **per-atom overlap policy** ρ : a choice of arbitration for same-footprint concurrent pairs (add-wins, remove-wins, larger-timestamp-wins, ...). Composite rc is computed pointwise; cross-component pairs are **Either** by frame. This is the separation-logic reading of the repository's empirical fact that every production rc is **Either** except on same-key pairs (**osRc**; the RGA's **Either**-everywhere; the mergeable queue's *absence* of any valid rc being the flag that its operations share one footprint — the whole sequence).

Certificates. An operation may carry an assertion over the id-heap — its *certificate* — required to hold at the issuing state (birth) and to be re-establishable at delivery. F0–F2 operations carry **emp**. The tombstone-free RGA's **Ins/Del** carry $\text{lseg}(\text{prefix} ++ [\text{anchor}])$: the recorded ancestor path *is* a list-segment assertion, and honest delivery *is* certificate validity. This is the design decision that replaces a global rely: what the environment may do is bounded by what certificates it must preserve, checked per footprint.

5 The Kernel

5.1 Grammar

atoms A	$::= \text{GSet } \tau$	grow-only set of τ (the free cell)
	$\text{MaxCell } (\tau, <)$	write-max register ($\perp$ -initialized)
	$\text{Agree } \tau$	write-once cell
terms D	$::= A^\rho$	(ATOM)
	$D_1 \otimes D_2$	(PROD)
	$\bigotimes_{k:K} D$	(MAP)
	$\text{Forget}[\mu]_{\nu, \iota, \nu_{\text{op}}} D$	(FORGET)
	$\text{Guard}_{I, \text{app}} D$	(GUARD)

Here ρ is an overlap policy (the atom's rc on same-footprint pairs); $\nu : \text{State}_D \rightarrow \text{NF}$ is a retraction with section $\iota : \text{NF} \rightarrow \text{State}_D$, $\nu \circ \iota = \text{id}$; ν_{op} is the action of the retraction on operations (identity except at F3, §6); and $\mu \in \{\text{cc}, \text{flat}, \text{wit}\}$ is the *join species*: which soundness route the owed Join Lemma takes (conditioned-commutation, flat/unconditional, or direct linearization witness). The species is a mode annotation, not a new judgment: the framework already ships all three as proved routes (the flat metatheorem; the conditioned metatheorem; the mergeable queue's direct witness `queue_ra_linearizable3`).

Remark 5.1 (Near-single-atom basis). *In principle the atom basis is GSet alone — MaxCell = Forget[flat](GSet) with $\nu = \text{max}$ (an F1 forgetting) and Agree is its degenerate write-once form. They are kept primitive as a pre-discharged library: their laws are proved once and derivations cite them.*

5.2 The typing judgment

$$D \vdash \langle \Sigma; \approx; O \rangle$$

Σ — the denoted MRDT signature; $\approx$ — the equivalence the judgment is up to; O — the multiset of *owed obligations*: leaf-level proof debt that a derivation leaves to the datatype author. Aletheia (§8) states: if $D \vdash \langle \Sigma; \approx; O \rangle$ and every obligation in O is discharged, then Σ is RA-linearizable up to $\approx$ (under honest delivery iff an F3 Forget or a Guard occurs in D), and $\Sigma \approx$ -refines its free expansion.

5.3 Rules

(Atom).

$$\frac{A^\rho \vdash \langle \Sigma_A \text{ with rc} = \rho \text{ on overlap, Either off; } (=); \{\text{atom laws of } A\} \rangle}{\text{ATOM}}$$

The atom laws (semilattice laws for GSet; totality of $<$ for MaxCell; single-writer for Agree) are library facts, discharged once.

(Prod).

$$\frac{D_1 \vdash \langle \Sigma_1; \approx_1; O_1 \rangle \quad D_2 \vdash \langle \Sigma_2; \approx_2; O_2 \rangle \quad K(D_1) \cap K(D_2) = \emptyset}{D_1 \otimes D_2 \vdash \langle \Sigma_1 \times \Sigma_2 \text{ pointwise; } \approx_1 \times \approx_2; O_1 \cup O_2 \rangle} \text{PROD}$$

No new obligation. This is the frame rule, and it is the one rule already fully mechanized: convergence by concatenation witness, one-sided safety, and the $\approx$ -quotient lift (`Metatheory/Product*.lean`; first real consumer PeritextTF, 1,064 lines for rich text). The side condition is syntactic: disjoint key-spaces, hence cross `rc = Either` by Proposition 4.1. The known semantic caution — reachability does not project through $\otimes$ — is *why* the rule owes a concatenation witness rather than a naive pairing; the witness is proved once, in the kit, not per instance.

(Map). $\otimes_{k:K} D$ — the keyed family: operations address one key, state is the finitely-supported product. Semantically an iterated $\otimes$ with uniform components; **no new obligation** beyond D ’s, *provided* each operation’s footprint is a single key (syntactic check). Owed to the mechanization: the pointwise lift as a once-only theorem (it exists today only implicitly, inside each set/map datatype’s own discharge).

(Forget) — the operator the calculus exists for.

$$\frac{D \vdash \langle \Sigma; \approx; O \rangle \quad \nu : \Sigma.\text{State} \rightarrow \text{NF} \quad \iota : \text{NF} \rightarrow \Sigma.\text{State} \quad \nu \circ \iota = \text{id}}{\text{Forget}[\mu]_{\nu, \iota, \nu_{\text{op}}} D \vdash \langle \Sigma_{\text{NF}}; \approx_{\nu}; O \cup O_F(\text{level}, \mu) \rangle} \text{FORGET}$$

where

$$\begin{aligned} \Sigma_{\text{NF}}.\text{init} &:= \nu(\Sigma.\text{init}), & \Sigma_{\text{NF}}.\text{do } s \ o &:= \nu(\Sigma.\text{do } (\iota s) (\nu_{\text{op}} o)), \\ \Sigma_{\text{NF}}.\text{mergeL } l \ a \ b &:= \nu(\Sigma.\text{mergeL } (\iota l) (\iota a) (\iota b)), & \approx_{\nu} &:= \text{the } \nu\text{-kernel quotient of } \approx. \end{aligned}$$

The owed obligation O_F is stratified by *how much ν forgets* — the ladder, §6. $\nu_{\text{op}} = \text{id}$ except at F3.

(Guard). $\text{Guard}_{I, \text{app}} D$ — the conditioning wrapper: invariant I on states, applicability `app` on (state, op). Denotation unchanged; the judgment gains a safety clause (I holds at every version) and the delivery assumption strengthens to honest (`app` at each application). Owed: **stability** — I is preserved by `do` of applicable operations and by `mergeL` of I -states arising in honest configurations — discharged by one of the two proved safety routes (causal-witness or escrow; both landed in the conditioning kits). The escrow instance: `app` consumes `tok(n)` resources whose total is bounded by I — the bounded counter is exactly this (`bc_version_inv`). **The rule must be allowed to refuse:** see §9 for FWW-mutex and the ungated BudgetCart budget, where refusal is the correct and already-established outcome.

6 The Forgetting Ladder

Stratify Forget by what the retraction destroys.

Conjecture 6.1 (C1: ladder strictness). *Each level’s obligations strictly contain the previous level’s. The inclusions are evident; strictness at each step is witnessed by the exemplar datatypes below (see also Open Question 10.6).*

F0 — no forgetting ($\nu = \text{id}$). Grow-only datatypes; merge is union; obligations: atom laws only. *Exemplars:* G-set, G-map, tombstoned RGA, Peritext (all components grow-only, pointwise union).

F1 — join-compatible forgetting. The kernel of ν is a semilattice congruence:

Obligation 6.2 (F1-law). $\nu(x \sqcup y) = \nu(\iota(\nu x) \sqcup \iota(\nu y))$ for all reachable x, y .

The normal forms then carry an induced *binary* join; the datatype remains a CRDT after forgetting, and no LCA appears. *Exemplars:* LWW register ($\nu = \text{keep-max}$: the max of a union is the max of maxes), FWW register (min-cell, the positive complement of the `lww_merge_needs_timestamps` kill-test), plausibly MVR ($\nu = \text{keep}$ the undominated antichain — *tentative, to validate by hand-derivation*, RQ1).

F2 — LCA-recoverable forgetting. ν is *not* a join-congruence: the join of images is not determined by the images. The lost information is recovered from the LCA. The obligation is **Eunoë**, stated at the only level of generality that survives Remark 2.1 (branches may share beyond the LCA):

Obligation 6.3 (Eunoë, the F2 obligation). *There is a normal-form merge $m(l, a, b)$ such that in every reachable coherent configuration, m at the version-DAG’s LCA-images equals the ν -image of the log-level union — inductively over the DAG. Concretely: the shipped VC bundle `CoreVCs3CD + FeasibleDeltaVCs3 + CDVC3` for Σ_{NF} , under join species μ .*

The shape of m is *inclusion-exclusion*: subtract from the branches what the LCA shows to be shared history. Three instances of the same formula:

datatype	normal form	$m(l, a, b)$
counter	event count	$a + b - l$
OR-set	live instances	$(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$ (= <code>osMergeL</code>)
monoid cell, general	fold of events	branch deltas re-applied to l

CDVC3 is the causal-delta law of this level, and its machine-checked independence (`cdvc3_not_derivable_from_core_delta`) is now read as: **the Eunoë law is a primitive axiom of the calculus, not derivable from the core and delta laws — the rule basis is minimal at F2.** *Exemplars:* IO-counter, PN-counter, OR-set, efficient OR-set (same free term, two retractions — the “efficient” variant is a *further* ν ; the calculus makes “efficient” a relation between terms, not a new datatype), EW-flag (tentative), AWPQ (tentative), and the mergeable queue at species `wit` (no `rc` exists — the enqueue clique shares the sequence footprint — so `cc` is structurally unavailable and the Join Lemma is the direct witness, exactly `queue_ra_linearizable3`).

F3 — certificate-carrying forgetting. ν destroys state that the *operations’ own semantics* reads: after forgetting, an operation’s meaning is no longer a function of the normal form it is applied to. Two things become necessary, and the calculus makes both explicit where the framework discovered them semantically:

1. $\nu_{\text{op}} \neq \text{id}$ — the retraction acts on operations: payloads are extended with certificates (§4) recording the destroyed context at birth, and `do` on normal forms consults the certificate.
2. **Honest delivery** — born-valid certificates, applicable at delivery — enters the judgment; it is consumed by nothing below F3.

Obligation: Eunoë *plus* honesty-threading (born accuracy + applicability, inductively over the DAG) *plus* correctness of ν_{op} (the rewritten operation on the normal form simulates the original operation on the free state, up to $\approx_\nu$). *Exemplar (the only one)*: the tombstone-free RGA. Free term: the tombstoned RGA (an F0 term!). ν : drop dead nodes, rehome children along recorded parent chains. Certificates: `lseg(prefix ++ [anchor])`. $\approx_\nu$: ghost payloads — and the eq-variant result (`RGA_Tombstone_Free_Eq_MRDT.lean`: on normal forms, $\approx$ *is* structural equality) is precisely the statement that $\approx_\nu$ is the ν -kernel quotient and nothing more. The impossibility of a prefix-free variant (`RGA_PrefixFree_Impossible.lean`) becomes a calculus theorem-shape: **F3 without certificates is underivable** (§9).

Orthogonal axis — Guard. Safety conditioning composes with any level: $\text{BC} = \text{Guard}_{\text{escrow}}(\text{F2 counter})$; $\text{BudgetCart} = \text{Guard}(\text{OR-set} \otimes \text{spend})$ with the budget theorem hypothesis-gated (OQ8 — the refusal is the framework's, and the calculus must reproduce it, §9).

Remark 6.4 (The ladder retro-dicts the difficulty gradient). *F0/F1 datatypes discharged with `dsimp+grind` almost entirely; F2 needed the conditioned metatheory; the single F3 datatype needed *HonestDelivery* and a bespoke end-to-end proof. The calculus did not choose this ordering — the proofs did.*

7 Worked Derivations

Component-by-component, per the house format. Certificates are **emp** unless stated.

7.1 LWW register — $\text{Forget}[\text{flat}]_{\text{keep-max}}(\text{GSet}(\mathbb{N} \times \mathbb{N} \times \tau))$ [F1]

component	free term	after Forget
Type	$\text{Finset}(ts \times \text{rid} \times \tau)$	$\text{Option}(ts \times \text{rid} \times \tau)$
init	$\emptyset$	none
do ($\text{wr } v$ at (t, r))	$\text{insert}(t, r, v)$	keep the $<$ -larger of old and (t, r, v)
merge	union	$\max l a b = \max a b$ (l drops out — F1)
rc	Either (overlap resolved by ts order)	same
Inv / app	$\top / \top$	$\top / \top$
$\approx$	$=$	$=$ (ν -kernel trivial on NF)
honesty	none	none

Obligation discharged: the F1-law $= \max(x \cup y) = \max(\max x, \max y)$. One line. The LCA vanishing from the merge *is the observable content of F1*.

7.2 Counter — $\text{Forget}[\text{flat}]_{\text{card}}(\text{GSet}(ts \times rid))$ [F2]

component	free term	after Forget
Type	$\text{Finset}(ts \times rid)$ (inc events)	$\mathbb{N}$
init	$\emptyset$	0
do (inc)	insert	+1
merge	union	$\mathbf{a} + \mathbf{b} - \mathbf{l}$
rc	Either	Either
$\approx$	=	=

Eunoë here is inclusion–exclusion on cardinality; l cannot be eliminated (the join of two numbers does not determine the union’s size) — the minimal possible F2, and the right first mechanization target. PN-counter = (this) $\otimes$ (this).

7.3 Efficient OR-set — $\text{Forget}[\text{cc}]_{\text{live}}(\otimes_{\text{inst}}(\text{GSet } 1 \otimes \text{GSet } 1))^{\text{add-wins}}$ [F2]

Free term: per instance $(ts, rid, \text{payload})$, an add-flag GSet and a remove-flag GSet (a keyed 2P pair). ν keeps instances with $\text{add} \wedge \neg \text{remove}$ — the live set. This lands verbatim on `ORSetCore.lean`:

component	value (post-Forget)	repo artifact
Type	$\text{Finset}(\mathbb{N} \times \mathbb{N} \times \beta)$	<code>OSState β</code>
init	$\emptyset$	<code>OSCore.init</code>
do	add: insert instance; rem k : filter key	<code>osUpdate</code>
merge	$(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$	<code>osMergeL</code>
rc	add-wins on same key, else Either	<code>osRc</code> — ρ is the same-key clause
$\approx$	=	—

Eunoë instance to check by hand: the live-set of a log-union against the inclusion–exclusion formula, DAG-inductively (the per-triple identity fails in general — Remark 2.1 — which is *why* the obligation is the VC bundle). The plain (tombstoned) OR-set is the *same free term with a weaker ν* ; the plain and efficient variants are two points on one Forget chain, and their separate 24-VC discharges in the repository are the two Eunoë instances.

7.4 Tombstone-free RGA — Forget[cc]_{drop+rehome; ν_{op}}(tombstoned RGA) [F3]

component	free term (F0, in repo as RGA_MRDT)	after Forget
Type	grow-only: nodes ($ts \mapsto elem \times anchor$) + tombstone set	live nodes only: $ts \mapsto (elem, anchor)$
do	Ins after x adds node; Del x adds tombstone	Ins $pre\ x$: insert with certificate; Del $pre\ x$: remove + rehome children to resolve (pre)
merge	pointwise union	climb-LCA rehoming of survivors (climb)
rc	Either	Either (order is read-side)
certificate	emp	lseg($pre \dashv\vdash [x]$) — the recorded ancestor path
$\approx$	=	ghost-payload quotient; on normal forms $\approx = =$ (eq-variant)
honesty	none	born accuracy + applicable delivery (HonestDelivery)

This is the derivation where the note’s one flagged surprise lives: ν_{op} is not optional — deletion on the normal form *edits surviving state* (children re-anchor), which no state-only retraction produces. Whether **Forget** is one operator with an op-action or two operators (state-quotient $\circ$ op-compiler) is Open Question 10.1. Everything owed here is already discharged *as an instance* (**RA_Lin.lean**, kernel-clean, one assumption = **HonestDelivery**); what Lethe owes is the rule-generic form.

7.5 Guarded exemplars, briefly

- **Bounded counter**: $\text{Guard}_{v \geq 0, \text{escrow}}$ over the PN counter of §6.2; stability via tokens; **bc_version_inv** is the discharged obligation.
- **BudgetCart** = $\text{Guard}_{\text{budget}}(\otimes_{\text{inst}} \text{OR-set} \otimes \text{derived-spend})$; convergence flows through the OR-set route; the *ungated* budget obligation is provably false (enumeration-dependence of vis-only folds) — the **GUARD** rule must refuse, matching **bcart_version_inv_gated** and OQ8.

7.6 The catalogue as Lethe terms (proposed; validation = RQ1)

datatype	proposed term	level	status of proposal
G-set / G-map	$\otimes$ GSet / $\otimes_k$ GSet	F0	clear
tombstoned RGA	$\otimes_{ts}$ (Agree $\otimes$ GSet) grow-only	F0	clear
Peritext	tombstoned RGA $\otimes$ GSet AnchorAttachment	F0	clear
LWW register	§6.1	F1	derived above
FWW register	Forget[flat] _{keep-min} (GSet)	F1	clear; Guard-mutex refused (§9)
MVR	Forget _{antichain} (GSet tagged writes)	F1/F2?	hand-derive: does the antichain ν satisfy F1, or does dominance need the LCA?
IO / PN counter	§6.2 ($\otimes$ for PN)	F2	derived above
OR-set / efficient	§6.3 (two ν 's)	F2	derived above
EW-flag	Forget(OR-set-of-enables)-like	F2?	hand-derive; the known-broken variant should <i>fail</i> a specific Eunoë instance — a calculus-level account of the Plausible counterexample
AWPQ	$\otimes$ (OR-set instance) + read-side max	F2?	hand-derive
mergeable queue	Forget[wit](Log) — no ρ exists	F2 ($\mu = \text{wit}$)	matches <code>queue_ra_linearizable3</code> ; the <i>absence</i> of an rc is predicted by the shared footprint
tombstone-free RGA	§6.4	F3	derived above
PeritextTF	(§6.4) $\otimes$ markstore	F3 $\otimes$ F0	matches <code>Peritext_Composed/</code> , the frame-rule consumer
BC / BudgetCart	§6.5	Guardo F2	derived above

8 Aletheia: the Soundness Theorem (Statement Owed)

Theorem 8.1 (Aletheia). *If $D \vdash \langle \Sigma; \approx; O \rangle$ and every obligation in O is discharged, then:*

1. (convergence) Σ is RA-linearizable up to $\approx$ at every version of every reachable coherent configuration — under honest delivery iff D contains an F3 Forget or a Guard, unconditionally otherwise;
2. (un-forgetting) the composite retraction ν_D assembled along the derivation exhibits $\llbracket D \rrbracket \approx$ -refining its free expansion — every reachable Σ -state is ν_D of a reachable free state, and queries factor through ν_D ;
3. (safety) every Guard_I node's I holds at every version, given the client honesty contract.

Proved by one induction on the typing derivation.

Case-by-case, what exists versus what is owed:

rule case	semantic lemma	status
ATOM	atom laws	exists piecemeal in instance discharges; owed: library form
PROD	concatenation witness + safety + $\approx$ -lift	exists (<code>Product*.lean</code>)
MAP	pointwise lift	owed (implicit in per-datatype proofs today)
FORGET F1	join-congruence $\Rightarrow$ CRDT	easy; owed
FORGET F2	Eunoë $\Rightarrow$ VC bundle $\Rightarrow$ JoinLemma3 $\Rightarrow$ RA-lin	exists (flat + conditioned metatheorems); owed: the transport along ν as a lemma rather than per-instance
FORGET F3	honesty-threading + ν_{op} simulation	exists as the RGA instance (<code>RA_Lin.lean</code>); owed: rule-generic form
GUARD	two safety routes	exists (causal-witness + escrow kits)

The honest summary: **the semantic content of every case except Map and the rule-generic F3 already exists kernel-checked; what does not exist is the grammar, the composite ν , and the single induction.** That is the mechanization gap `oq:proglogic` names, now with a precise worklist.

9 What the Calculus Predicts and Refuses

A logic earns its keep by refusing derivations, and each refusal below is *already a kernel-checked or documented impossibility* — the calculus turns them from post-hoc discoveries into syntactic non-derivability:

1. **No F3 without certificates.** A prefix-free tombstone-free RGA is not a Lethe term (ν_{op} has no certificate to consult), and the framework proved the corresponding datatype unverifiable (`RGA_PrefixFree_Impossible.lean`).
2. **No Guard-mutex from merges.** $\text{Guard}_{\text{unset} \Rightarrow \text{exclusive}}$ over FWW fails stability (“unset” is not stable under concurrent honest extension) — the documented reason the FWW register is a reservation, never a mutex. The GUARD rule’s stability side-condition refuses exactly this.
3. **No ungated BudgetCart budget.** The two-sided safety obligation is provably false (enumeration-dependence); GUARD derives only the gated form (`bcart_version_inv_gated`, OQ8).
4. **No rc for the queue.** Concurrent enqueues share the sequence footprint; Proposition 4.1 makes ρ unavailable, forcing $\mu = \text{wit}$ — which is exactly the route the mechanized proof took.

And the constructive test (**the falsifiable success criterion**): build a datatype *not in the catalogue* — a bounded OR-set ($\text{Guard}_{\text{escrow}}$ over §6.3) or an LWW-map with eviction ($\otimes_k$ over §6.1 with a Forget dropping evicted keys) — where the only manual proof is the leaf Eunoë/F1 instance. If the kernel cannot produce something new, it is a taxonomy; if it can, it is a calculus.

10 Open Questions

Open Question 10.1 (OQ-L1: Forget — one operator or two?). *F3 forces an op-action ν_{op} . Is Forget a single operator with a (usually trivial) op-action, or a state-quotient composed with an op-compiler? The RGA hand-derivation (§7.4) suggests the compiler view: certificates are compiled-in preconditions. Decide on paper before mechanizing the FORGET rule.*

Open Question 10.2 (OQ-L2: exactness of Eunoë). *Is the F2 obligation equivalent to the shipped bundle `CoreVCs3CD + FeasibleDeltaVCs3 + CDVC3`, or merely sufficient? The CDVC3 countermodel gives the “not less” direction; the “not more” direction is open. If exact, the framework is the soundness theorem of one operator — the paper’s headline.*

Open Question 10.3 (OQ-L3: SL sufficiency at F3). *Do footprint certificates + per-key invariants fully replace a global rely — i.e., can **HonestDelivery** be derived from per-op certificate validity, with the RGA `lseg` as the test? Expected yes for F0–F2 (vacuous), genuinely open at F3.*

Open Question 10.4 (OQ-L4: efficiency as a judgment). *Forget chains order variants of one datatype (plain vs. efficient OR-set). Make “efficient” formal: normal-form size / merge cost versus the free term. Even semi-formal, it justifies the word in the thesis.*

Open Question 10.5 (OQ-L5: completeness). *Which RA-linearizable MRDTs are not Lethe terms? The queue needed a mode, not a new operator — evidence the mode lattice $\{\text{cc}, \text{flat}, \text{wit}\}$ might suffice; the MVR and AWPQ hand-derivations (§7.6) are the next stress tests.*

Open Question 10.6 (OQ-L6: ladder strictness — Conjecture 6.1). *Each level’s obligations strictly contain the previous. Strictness $F1 \subsetneq F2$ is witnessed by the counter (l does not drop out); $F2 \subsetneq F3$ by the prefix-free impossibility. $F0 \subsetneq F1$ needs a datatype whose $\nu = \text{id}$ fails F1 vacuously — trivial, but state it.*

11 Relation to Existing Artifacts (crosswalk)

Lethe piece	existing artifact
PROD rule	<code>Metatheory/Product*.lean</code> (concatenation witness, safety, $\approx$ -lift)
F2 obligation	<code>Framework/VC_Set.lean</code> bundle; minimality: <code>Refutations/CD_Not_Derivable_Ternary.lean</code>
F3 instance	<code>MRDT_Instances/RGA/RA_Lin.lean</code> ; $\approx$ -collapse: <code>RGA_Tombstone_Free_Eq_MRDT.lean</code>
$\mu = \text{wit}$	<code>MRDT_Instances/MergeableQueue/queue_ra_linearizable3</code>
GUARD routes	conditioning kits (causal-witness, escrow); <code>bc_version_inv</code> , <code>bcart_version_inv_gated</code>
refusals	<code>RGA_PrefixFree_Impossible.lean</code> ; FWW reservation documentation; OQ8
§6.3 verbatim	<code>MRDT_Instances/ORSetCore/ORSetCore.lean</code> (<code>osUpdate</code> , <code>osMergeL</code> , <code>osRc</code>)

Scope limit, stated once. Aletheia certifies convergence to the datatype’s *own* fold up to $\approx$ — not intent (the `oq:linsec` limit; the Peritext mark-leak is the canonical example). A spec layer per atom (abstract state + per-op refinement) is future work and deliberately outside this kernel.